

SeatWise – Software Requirements Specification

Version: 1.0
Date: January 22, 2026
Project Type: Event Ticketing Microservices Platform
Target Market: East Africa (Rwanda, Kenya, Uganda)
Status: Ready for Development

Document Control

Version	Date	Author	Changes
1.0	January 22, 2026	Development Team	Initial release integrating architecture design and requirements

Table of Contents

- 1. Executive Summary
 - 2. System Overview
 - 3. User Roles and Permissions
 - 4. Functional Requirements
 - 5. Critical System Workflows
 - 6. Database Schema Specifications
 - 7. Non-Functional Requirements
 - 8. Technology Stack
 - 9. System Constraints and Assumptions
 - 10. Interface Requirements
 - 11. Data Migration and Seeding
 - 12. Testing Requirements
 - 13. Deployment Architecture
 - 14. Future Enhancements
 - 15. Glossary of Terms
 - 16. Appendices
 - 17. Approval and Sign-Off
-

1. Executive Summary

SeatWise is a distributed event ticketing platform built on microservices architecture, enabling users to discover events, reserve seats with real-time availability tracking, and complete transactions through mobile money payment systems. The system demonstrates enterprise-grade backend patterns including service-oriented architecture, distributed caching, asynchronous messaging, and polyglot persistence strategies.

1.1 Business Context

The platform addresses the event ticketing needs of the East African market, where mobile money payment infrastructure is ubiquitous and real-time seat reservation is critical for high-demand events. The system supports venues ranging from small conference halls to large stadiums with complex seating arrangements.

1.2 Key Differentiators

- **Microservices Architecture:** Independent services with clear boundaries and separate data stores
 - **Polyglot Persistence:** MongoDB for flexible event/venue schemas, PostgreSQL for transactional integrity
 - **Mobile Money Integration:** Native support for MTN Mobile Money, Airtel Money payment gateways
 - **Real-Time Seat Locking:** Redis-backed temporary reservations with automatic expiry
 - **Multi-Channel Notifications:** Email and SMS delivery for booking confirmations and tickets
-

2. System Overview

2.1 System Architecture

SeatWise employs a microservices architecture pattern with the following core components:

2.1.1 Architecture Pattern

- **Type:** Microservices Architecture with API Gateway
- **Communication:** Synchronous REST APIs for request-response, asynchronous message broker for events
- **Data Strategy:** Polyglot persistence with service-specific databases
- **Scalability:** Horizontal scaling of stateless services

2.1.2 System Components

Entry Point:

- **API Gateway** (Spring Cloud Gateway / Nginx): Single entry point for all client requests, handles routing, authentication validation, and rate limiting

Core Services:

1. User Service

- Database: PostgreSQL
- Responsibilities: User registration, authentication, JWT token generation, role management, profile operations

2. Event Service

- Database: MongoDB
- Responsibilities: Event lifecycle management, venue configuration, seat map management, seat inventory tracking

3. Booking Service

- Database: PostgreSQL
- Responsibilities: Reservation orchestration, payment processing, ticket generation, transaction management

4. Notification Service

- Database: MongoDB
- Responsibilities: Email/SMS delivery, notification logging, PDF ticket generation

Infrastructure Components:

- **Redis Cache:** Session management, temporary seat locks, notification caching
- **Message Broker:** RabbitMQ or Apache Kafka for asynchronous event distribution
- **Payment Gateway:** Mobile Money API integration (MTN MoMo, Airtel Money)

2.2 Communication Patterns

Synchronous Communication (REST APIs)

- Client ↔ API Gateway
- API Gateway ↔ All Services
- Booking Service ↔ Event Service (seat availability checks)
- Booking Service ↔ User Service (user validation)

Asynchronous Communication (Message Broker)

- Booking Service → Notification Service (booking events)
 - User Service → Notification Service (authentication events)
 - Redis Expiry Events → Booking Service (timeout handling)
-

3. User Roles and Permissions

The system shall support three distinct user roles with hierarchical permission structures.

Table 3.1: User Roles and Access Control

Role	Description	Permissions
USER	Standard authenticated user	Register, authenticate, browse events, reserve seats, initiate payments, view booking history, receive notifications
ADMIN	Event administrator and platform manager	All USER permissions plus: create events, configure venues, manage seat maps, update event details, cancel bookings, view analytics
SYSTEM	Automated background processes	Release expired reservations, trigger scheduled notifications, process payment callbacks, cleanup stale data

3.1 Role Assignment

- R-ROLE-001:** All new user registrations shall default to the USER role.
- R-ROLE-002:** ADMIN role assignment shall require a pre-shared administrative secret key provided during registration.
- R-ROLE-003:** SYSTEM role operations shall be executed exclusively by authenticated background processes and scheduled jobs.
- R-ROLE-004:** The system shall enforce role-based access control at the API Gateway level before routing requests to downstream services.

4. Functional Requirements

4.1 API Gateway

The API Gateway shall serve as the single entry point for all client requests and enforce cross-cutting concerns.

4.1.1 Functional Capabilities

- FR-GW-001:** The API Gateway shall route incoming requests to appropriate microservices based on URL path patterns.
- FR-GW-002:** The API Gateway shall validate JWT tokens for all protected endpoints before forwarding requests.
- FR-GW-003:** The API Gateway shall implement rate limiting of 100 requests per minute per client IP address.

FR-GW-004: The API Gateway shall return standardized error responses with appropriate HTTP status codes.

FR-GW-005: The API Gateway shall add correlation IDs to all requests for distributed tracing.

4.2 User Service

The User Service shall manage user identity, authentication, and authorization.

4.2.1 Endpoints

Endpoint	Method	Description	Authorization
/api/users/register	POST	Create new user account	Public
/api/auth/login	POST	Authenticate user and issue JWT	Public
/api/users/profile	GET	Retrieve authenticated user profile	USER, ADMIN
/api/users/profile	PUT	Update user profile information	USER, ADMIN
/api/admin/register	POST	Create administrative user	Public (requires admin secret)
/api/users/{id}	GET	Retrieve user details by ID	ADMIN

4.2.2 Functional Capabilities

- FR-US-001:** The system shall authenticate users using JWT-based token validation with HS256 signing algorithm.
- FR-US-002:** The system shall issue JWT tokens with 24-hour expiration containing user ID and role claims.
- FR-US-003:** The system shall hash user passwords using bcrypt algorithm with work factor of 12 before database storage.
- FR-US-004:** The system shall validate the administrative secret key during ADMIN registration and reject invalid keys with HTTP 403 response.
- FR-US-005:** The system shall enforce unique username constraints and return HTTP 409 for duplicate registration attempts.
- FR-US-006:** The system shall publish `user.login` events to the message broker upon successful authentication.
- FR-US-007:** The system shall store user credentials in PostgreSQL database with ACID transaction guarantees.
-

4.3 Event Service

The Event Service shall manage event information, venue configurations, and seat inventory.

4.3.1 Endpoints

Endpoint	Method	Description	Authorization
<code>/api/events</code>	GET	Retrieve list of all active events	Public
<code>/api/events/{id}</code>	GET	Retrieve event details and seat map	Public
<code>/api/events</code>	POST	Create new event with venue configuration	ADMIN
<code>/api/events/{id}</code>	PUT	Update event details or seat map	ADMIN
<code>/api/events/{id}</code>	DELETE	Delete event and associated seats	ADMIN
<code>/api/events/{id}/seats/availability</code>	GET	Check availability of specific seats	USER, ADMIN
<code>/api/events/{id}/seats/lock</code>	POST	Lock seat for temporary reservation	Booking Service
<code>/api/events/{id}/seats/confirm</code>	POST	Confirm seat booking permanently	Booking Service
<code>/api/events/{id}/seats/release</code>	POST	Release locked or reserved seat	Booking Service, SYSTEM

4.3.2 Functional Capabilities

FR-ES-001: The system shall store event data in MongoDB to support flexible venue schemas with nested seat map configurations.

FR-ES-002: The system shall support complex seating arrangements including sections, rows, and seat numbering patterns.

FR-ES-003: The system shall maintain seat status using four discrete states: AVAILABLE, LOCKED, BOOKED, SOLD.

FR-ES-004: The system shall validate seat lock requests and return HTTP 409 if the seat is not in AVAILABLE status.

FR-ES-005: The system shall record lock timestamp when transitioning seat to LOCKED status.

FR-ES-006: The system shall prevent seat status modifications once a seat reaches SOLD status except by ADMIN override.

FR-ES-007: The system shall cascade delete all associated seat records when an event is deleted.

FR-ES-008: The system shall return event listings sorted by event date in ascending order by default.

FR-ES-009: The system shall include venue metadata in event responses including capacity, layout type, and accessibility features.

4.4 Booking Service

The Booking Service shall orchestrate the seat reservation workflow, payment processing, and ticket generation.

4.4.1 Endpoints

Endpoint	Method	Description	Authorization
<code>/api/bookings/reserve</code>	POST	Initiate seat reservation with timer	USER
<code>/api/bookings/{id}/payment</code>	POST	Process payment and confirm booking	USER (owner)
<code>/api/bookings/{id}</code>	GET	Retrieve booking details and status	USER (owner), ADMIN
<code>/api/bookings/user/history</code>	GET	Retrieve user's booking history	USER, ADMIN
<code>/api/bookings/{id}/cancel</code>	POST	Cancel pending or confirmed booking	USER (owner), ADMIN
<code>/api/bookings/expired/release</code>	POST	Release expired reservations	SYSTEM

4.4.2 Functional Capabilities

FR-BS-001: The system shall implement a two-phase booking workflow: reserve with timeout, then payment confirmation.

FR-BS-002: The system shall invoke Event Service REST API to lock seats when reservation is initiated.

FR-BS-003: The system shall create Redis cache entry with 5-minute TTL when seat is locked, storing booking ID and seat reference.

FR-BS-004: The system shall return reservation details including countdown timer value and payment instructions.

FR-BS-005: The system shall integrate with Mobile Money payment gateway APIs (MTN MoMo, Airtel Money) for transaction processing.

FR-BS-006: The system shall validate payment callback signatures to prevent fraudulent confirmations.

FR-BS-007: Upon successful payment callback, the system shall:

- Update booking status to CONFIRMED in PostgreSQL
- Invoke Event Service to transition seat status from LOCKED to SOLD

- Publish `booking.confirmed` event to message broker
- Generate unique booking confirmation code

FR-BS-008: Upon payment failure or timeout, the system shall:

- Update booking status to FAILED or EXPIRED
- Invoke Event Service to release seat (transition to AVAILABLE)
- Publish `booking.failed` event to message broker

FR-BS-009: The system shall listen for Redis key expiry events and trigger seat release for expired reservations.

FR-BS-010: The system shall generate QR code containing booking ID and validation hash for ticket verification.

FR-BS-011: The system shall track booking state using five discrete statuses: PENDING, PAYMENT_INITIATED, CONFIRMED, FAILED, EXPIRED.

FR-BS-012: The system shall store booking records in PostgreSQL with foreign key constraints to users and events.

4.5 Notification Service

The Notification Service shall deliver multi-channel notifications and generate digital tickets.

4.5.1 Endpoints

Endpoint	Method	Description	Authorization
<code>/api/notifications</code>	GET	Retrieve user notification history	USER, ADMIN
<code>/api/notifications/{id}/read</code>	PUT	Mark notification as read	USER (owner)
<code>/api/notifications/unread/count</code>	GET	Get count of unread notifications	USER, ADMIN

4.5.2 Functional Capabilities

FR-NS-001: The system shall consume `booking.confirmed` events from message broker and trigger notification delivery.

FR-NS-002: The system shall consume `booking.failed` events and send failure notifications to users.

FR-NS-003: The system shall consume `user.login` events and send login confirmation notifications.

FR-NS-004: The system shall generate PDF tickets containing:

- Event details (name, date, time, venue)
- Seat information (section, row, number)
- QR code for verification
- Booking confirmation code
- User details

FR-NS-005: The system shall deliver notifications through multiple channels:

- Email via SendGrid API
- SMS via Twilio API
- In-app notification stored in MongoDB

FR-NS-006: The system shall attach PDF ticket to confirmation emails for CONFIRMED bookings.

FR-NS-007: The system shall cache notification delivery status in Redis for 24 hours to support retry logic.

FR-NS-008: The system shall log all notification attempts in MongoDB including timestamp, channel, status, and error details.

FR-NS-009: The system shall retry failed email/SMS deliveries up to 3 times with exponential backoff (1 min, 5 min, 15 min).

FR-NS-010: The system shall support four notification types: BOOKING_CONFIRMED, BOOKING_FAILED, PAYMENT_PENDING, USER_LOGIN.

FR-NS-011: The system shall track notification read status using two discrete states: UNREAD, READ.

5. Critical System Workflows

5.1 Seat Reservation and Payment Flow

This workflow describes the end-to-end process from seat selection to confirmed booking with ticket delivery.

Phase 1: Seat Reservation (5-Minute Timer Starts)

1. **User Action:** User selects seat(s) from stadium map interface and clicks "Book Now"
2. **API Gateway:** Validates JWT token, routes request to Booking Service
3. **Booking Service:**
 - Validates user has no pending reservations for same event
 - Calls Event Service REST API: `POST /events/{id}/seats/lock`
4. **Event Service:**
 - Queries MongoDB for seat document

- Validates seat status is AVAILABLE
- Updates seat status to LOCKED with lock timestamp
- Returns success response with seat details

5. Booking Service:

- Creates booking record in PostgreSQL with status PENDING
- Creates Redis cache entry: `reservation:{bookingId}` with 5-minute TTL
- Returns booking ID and 300-second countdown to client

6. Client UI: Displays order summary screen with countdown timer and payment options

Phase 2: Payment Processing

7. User Action: User enters mobile money phone number and clicks "Pay Now"

8. Booking Service:

- Validates reservation has not expired (checks Redis key exists)
- Updates booking status to PAYMENT_INITIATED
- Calls Mobile Money API to initiate transaction

9. Mobile Money Gateway:

- Sends USSD prompt to user's phone
- User enters PIN on mobile device
- Processes payment transaction

Phase 3: Payment Callback and Confirmation

10. Payment Gateway: Sends HTTP callback to `POST /api/bookings/{id}/payment/callback`

11. Booking Service:

- Validates callback signature for authenticity
- If payment successful:
 - Updates booking status to CONFIRMED in PostgreSQL
 - Removes Redis reservation key (no longer needed)
 - Calls Event Service: `POST /events/{id}/seats/confirm` to mark SOLD
 - Publishes `booking.confirmed` event to RabbitMQ
 - Returns confirmation response to client
- If payment failed:
 - Updates booking status to FAILED
 - Calls Event Service: `POST /events/{id}/seats/release` to unlock
 - Publishes `booking.failed` event to RabbitMQ

12. Notification Service:

- Consumes `booking.confirmed` event from message queue
- Generates PDF ticket with QR code
- Sends email via SendGrid with ticket attachment
- Sends SMS via Twilio with confirmation code
- Stores notification record in MongoDB

13. **Client UI:** Displays success screen with booking confirmation code

Phase 4: Timeout Scenario (User Abandons Payment)

Alternative Path: If user does not complete payment within 5 minutes:

1. **Redis:** Key `reservation:{bookingId}` expires, triggers expiry event
 2. **Booking Service:**
 - Listens to Redis expiry channel
 - Detects expired reservation
 - Updates booking status to EXPIRED
 - Calls Event Service: `POST /events/{id}/seats/release`
 3. **Event Service:** Updates seat status from LOCKED to AVAILABLE
 4. **Notification Service:**
 - Consumes `booking.expired` event
 - Sends notification: "Your reservation expired. The seat is now available."
 5. **Client UI:** If user still on page, displays "Reservation expired" message
-

5.2 User Authentication and Login Notification Flow

1. **User Action:** User enters email and password on login screen
2. **API Gateway:** Routes request to User Service
3. **User Service:**
 - Retrieves user record from PostgreSQL by username
 - Validates password hash using bcrypt comparison
 - If valid:
 - Generates JWT token with claims: `{userId, role, exp: 24h}`
 - Publishes `user.login` event to message broker
 - Returns JWT token to client
 - If invalid: Returns HTTP 401 Unauthorized

4. **Client:** Stores JWT in localStorage/sessionStorage
 5. **Notification Service:**
 - Consumes `user.login` event
 - Sends email: "Login detected from [Device/Location]"
 - Stores USER_LOGIN notification in MongoDB
 6. **Subsequent Requests:** Client includes JWT in `Authorization: Bearer {token}` header
-

5.3 Event Creation and Seat Map Configuration (Admin Flow)

1. **Admin Action:** Admin fills event creation form with venue details
2. **API Gateway:** Validates JWT, confirms ADMIN role, routes to Event Service
3. **Event Service:**
 - Validates event date is in future
 - Creates event document in MongoDB with structure:

```
json
{
  "id": "uuid",
  "name": "Concert 2026",
  "date": "2026-06-15",
  "venue": {
    "name": "Kigali Arena",
    "capacity": 10000,
    "layout": "stadium"
  },
  "sections": [
    {
      "name": "VIP",
      "rows": 10,
      "seatsPerRow": 20,
      "price": 50000,
      "seats": [
        { "number": "A1", "status": "AVAILABLE" },
        { "number": "A2", "status": "AVAILABLE" }
      ]
    }
  ]
}
```

- Returns event ID and confirmation

4. **Admin UI:** Displays success message and event preview

6. Database Schema Specifications

6.1 User Service Database (PostgreSQL)

6.1.1 Users Table

Table Name: `users`

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique user identifier
username	VARCHAR(255)	NOT NULL, UNIQUE	User login name (email format)
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
full_name	VARCHAR(500)	NOT NULL	User display name
phone_number	VARCHAR(20)	NULL	Mobile number for payment
role	VARCHAR(20)	NOT NULL, CHECK (role IN ('USER', 'ADMIN'))	Authorization role
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Last profile update
last_login	TIMESTAMP	NULL	Last successful authentication

Indexes:

- `idx_users_username` on (username)
 - `idx_users_role` on (role)
-

6.2 Event Service Database (MongoDB)

6.2.1 Events Collection

Collection Name:

events

Document Schema:

json

```
{
  "_id": "ObjectId",
  "eventId": "UUID (indexed)",
  "name": "String (required)",
  "title": "String (required)",
  "description": "String",
  "date": "ISODate (required)",
  "time": "String (HH:mm format)",
  "venue": {
    "name": "String",
    "address": "String",
    "city": "String",
    "capacity": "Number",
    "layoutType": "String (stadium|theater|conference)"
  },
  "sections": [
    {
      "sectionId": "UUID",
      "name": "String (VIP, General, etc.)",
      "price": "Number (RWF)",
      "totalSeats": "Number",
      "availableSeats": "Number",
      "rows": [
        {
          "rowLabel": "String (A, B, C)",
          "seats": [
            {
              "seatId": "UUID",
              "seatNumber": "String (A1, A2)",
              "status": "String (AVAILABLE|LOCKED|SOLD)",
              "lockedBy": "UUID (userId, null if available)",
              "lockedAt": "ISODate (null if not locked)",
              "bookedBy": "UUID (userId, null if not booked)",
              "bookingId": "UUID (null if not booked)"
            }
          ]
        }
      ]
    }
  ],
  "status": "String (DRAFT|PUBLISHED|CANCELLED|COMPLETED)",
  "createdBy": "UUID (admin userId)",
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

Indexes:

- `{eventId: 1}` - Unique index
 - `{date: 1, status: 1}` - Compound index for event listings
 - `{"sections.rows.seats.seatId": 1}` - For seat lookups
-

6.3 Booking Service Database (PostgreSQL)

6.3.1 Bookings Table

Table Name: `bookings`

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique booking identifier
user_id	UUID	NOT NULL, FOREIGN KEY → users(id)	Booking user reference
event_id	UUID	NOT NULL	Reference to event in Event Service
seat_ids	JSONB	NOT NULL	Array of seat UUIDs
total_amount	DECIMAL(10,2)	NOT NULL	Total payment amount (RWF)
status	VARCHAR(30)	NOT NULL, CHECK (status IN ('PENDING', 'PAYMENT_INITIATED', 'CONFIRMED', 'FAILED', 'EXPIRED', 'CANCELLED'))	Current booking status
confirmation_code	VARCHAR(20)	NULL, UNIQUE	Unique booking reference
reserved_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Reservation timestamp
payment_initiated_at	TIMESTAMP	NULL	Payment start time
confirmed_at	TIMESTAMP	NULL	Confirmation timestamp
expires_at	TIMESTAMP	NOT NULL	Reservation expiry (reserved_at + 5 min)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Last update

Indexes:

- `idx_bookings_user_id` on (user_id)
- `idx_bookings_event_id` on (event_id)
- `idx_bookings_status` on (status)
- `idx_bookings_confirmation_code` on (confirmation_code)

- `idx_bookings_expires_at` on `(expires_at)` WHERE status = 'PENDING'

6.3.2 Transactions Table

Table Name: `payment_transactions`

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Transaction identifier
booking_id	UUID	NOT NULL, FOREIGN KEY → bookings(id)	Associated booking
payment_method	VARCHAR(50)	NOT NULL	MTN_MOMO, AIRTEL_MONEY
phone_number	VARCHAR(20)	NOT NULL	Payment phone number
amount	DECIMAL(10,2)	NOT NULL	Transaction amount
gateway_reference	VARCHAR(255)	NULL	External transaction ID
status	VARCHAR(30)	NOT NULL	INITIATED, SUCCESS, FAILED
callback_data	JSONB	NULL	Full gateway response
initiated_at	TIMESTAMP	NOT NULL	Payment initiation
completed_at	TIMESTAMP	NULL	Success/failure time

6.4 Notification Service Database (MongoDB)

6.4.1 Notifications Collection

Collection Name: `notifications`

Document Schema:

```
json
```

```

{
  "_id": "ObjectId",
  "notificationId": "UUID (indexed)",
  "userId": "UUID (indexed, required)",
  "type": "String (BOOKING_CONFIRMED|BOOKING_FAILED|PAYMENT_PENDING|USER_LOGIN)",
  "channels": {
    "email": {
      "sent": "Boolean",
      "sentAt": "ISODate",
      "messageId": "String (provider reference)",
      "error": "String (null if successful)"
    },
    "sms": {
      "sent": "Boolean",
      "sentAt": "ISODate",
      "messageId": "String",
      "error": "String"
    },
    "inApp": {
      "delivered": "Boolean",
      "readAt": "ISODate"
    }
  },
  "content": {
    "subject": "String",
    "message": "String",
    "metadata": {
      "bookingId": "UUID (if applicable)",
      "eventName": "String",
      "confirmationCode": "String"
    }
  },
  "attachments": [
    {
      "type": "String (PDF_TICKET)",
      "url": "String (S3/storage URL)",
      "filename": "String"
    }
  ],
  "status": "String (UNREAD|READ)",
  "priority": "String (LOW|NORMAL|HIGH)",
  "createdAt": "ISODate",
  "readAt": "ISODate (null if unread)"
}

```

Indexes:

- `{userId: 1, createdAt: -1}` - User notification history
- `{notificationId: 1}` - Unique index
- `{status: 1, createdAt: -1}` - Unread notifications query

6.4.2 Notification Logs Collection

Collection Name: `notification_logs`

Document Schema:

```
json
{
  "_id": "ObjectId",
  "notificationId": "UUID",
  "channel": "String (email|sms)",
  "provider": "String (SendGrid|Twilio)",
  "attempt": "Number (1, 2, 3)",
  "success": "Boolean",
  "responseCode": "String",
  "responseBody": "String",
  "errorMessage": "String (null if success)",
  "attemptedAt": "ISODate"
}
```

7. Non-Functional Requirements

7.1 Performance Requirements

NFR-PERF-001: The system shall complete seat availability checks within 200 milliseconds under normal load (up to 500 concurrent users).

NFR-PERF-002: The system shall process seat reservation requests (lock operation) within 500 milliseconds including database updates.

NFR-PERF-003: The API Gateway shall route requests to downstream services with latency not exceeding 50 milliseconds.

NFR-PERF-004: MongoDB queries for event details with embedded seat maps shall complete within 300 milliseconds for events with up to 10,000 seats.

NFR-PERF-005: Redis cache operations for notification retrieval shall complete within 10 milliseconds.

NFR-PERF-006: The system shall support a minimum throughput of 100 booking transactions per second during peak load.

NFR-PERF-007: Payment callback processing shall complete within 1 second to prevent gateway timeouts.

7.2 Scalability Requirements

NFR-SCALE-001: All microservices shall be designed as stateless components supporting horizontal scaling through container orchestration.

NFR-SCALE-002: The Event Service shall support events with seating capacities up to 50,000 seats without performance degradation.

NFR-SCALE-003: The system shall handle 10,000 concurrent active reservations (seats in LOCKED status) across all events.

NFR-SCALE-004: MongoDB sharding shall be supported for Event Service database when event count exceeds 100,000 documents.

NFR-SCALE-005: The Notification Service shall process message queue events at a rate of at least 500 notifications per minute.

NFR-SCALE-006: Redis cache shall support 100,000 concurrent reservation keys with sub-millisecond expiry event processing.

7.3 Reliability Requirements

NFR-REL-001: The system shall prevent double-booking scenarios with 100% consistency guarantee through seat status validation before lock operations.

NFR-REL-002: The system shall automatically release expired seat reservations within 10 seconds of Redis key expiry event.

NFR-REL-003: Database transactions for booking confirmation shall follow ACID properties with rollback on payment failure.

NFR-REL-004: The message broker shall guarantee at-least-once delivery for all notification events with durable queue persistence.

NFR-REL-005: The system shall maintain booking data integrity across service boundaries through compensating transactions on failure scenarios.

NFR-REL-006: Payment gateway integration shall implement idempotency for callback requests using unique transaction identifiers.

7.4 Availability Requirements

NFR-AVAIL-001: The system shall maintain 99.5% uptime during business hours (08:00-23:00 EAT).

NFR-AVAIL-002: Individual microservice failures shall not cascade to other services; degraded functionality is acceptable.

NFR-AVAIL-003: Redis cache unavailability shall not prevent booking operations; the system shall fall back to direct database queries.

NFR-AVAIL-004: The API Gateway shall implement circuit breaker patterns with 5-second timeout for downstream service calls.

NFR-AVAIL-005: All services shall expose `/health` and `/ready` endpoints for container orchestration health checks.

NFR-AVAIL-006: The system shall implement graceful degradation: if Notification Service is unavailable, bookings shall still complete with delayed notifications.

7.5 Security Requirements

NFR-SEC-001: All API endpoints shall require HTTPS/TLS 1.3 encryption for data in transit.

NFR-SEC-002: JWT tokens shall be signed using HS256 algorithm with a 256-bit secret key rotated every 90 days.

NFR-SEC-003: The system shall enforce role-based access control at API Gateway level before routing to services.

NFR-SEC-004: User passwords shall be hashed using bcrypt with work factor 12 before storage.

NFR-SEC-005: The administrative secret key for ADMIN registration shall be at least 32 characters and stored in environment variables.

NFR-SEC-006: Payment gateway callback endpoints shall validate request signatures to prevent spoofing attacks.

NFR-SEC-007: The system shall implement rate limiting of 10 login attempts per IP address per minute to prevent brute force attacks.

NFR-SEC-008: Sensitive data in logs (passwords, payment details) shall be masked or excluded from application logs.

NFR-SEC-009: Database connections shall use connection pooling with encrypted credentials stored in secrets management systems.

NFR-SEC-010: Cross-Origin Resource Sharing (CORS) shall be configured to allow requests only from authorized frontend domains.

7.6 Maintainability Requirements

NFR-MAINT-001: Each microservice shall maintain independent database schemas allowing autonomous deployment and schema evolution.

NFR-MAINT-002: All REST APIs shall be documented using OpenAPI 3.0 specification with example requests and responses.

NFR-MAINT-003: The system shall implement structured logging with JSON format including correlation IDs for distributed request tracing.

NFR-MAINT-004: Service-to-service communication shall use versioned API contracts (e.g., `/api/v1/events`) to support backward compatibility.

NFR-MAINT-005: Database migration scripts shall be version-controlled and support rollback for schema changes.

NFR-MAINT-006: Code coverage for unit tests shall exceed 70% for business logic components.

NFR-MAINT-007: Each service shall include README documentation describing setup, configuration, and deployment procedures.

7.7 Usability Requirements

NFR-USE-001: The countdown timer on the reservation screen shall update every second to reflect remaining time.

NFR-USE-002: The system shall display seat availability in real-time with visual indicators (green=available, orange=locked by others, red=sold).

NFR-USE-003: Error messages shall be user-friendly and actionable (e.g., "This seat was just booked by another user. Please select a different seat.").

NFR-USE-004: Mobile money payment instructions shall be displayed in the user's selected language (English, Kinyarwanda, French).

NFR-USE-005: PDF tickets shall be formatted for A4 paper size and mobile screen viewing.

7.8 Compliance and Localization Requirements

NFR-COMP-001: The system shall comply with Rwanda's Data Protection and Privacy Law regarding user data storage and processing.

NFR-COMP-002: Financial transaction records shall be retained for 7 years in compliance with tax regulations.

NFR-COMP-003: Payment amounts shall be displayed in Rwandan Francs (RWF) with proper currency formatting (e.g., "50,000 RWF").

NFR-COMP-004: Timestamps shall be stored in UTC and displayed in East Africa Time (EAT, UTC+3) on user interfaces.

NFR-COMP-005: The system shall support mobile money providers licensed to operate in Rwanda (MTN MoMo, Airtel Money).

8. Technology Stack

Table 8.1: System Components and Technologies

Component	Technology	Version	Justification
API Gateway	Spring Cloud Gateway	4.x	Native Spring Boot integration, dynamic routing, circuit breaker support
User Service	Spring Boot	3.2.x	Mature framework with excellent security libraries (Spring Security)
Event Service	Spring Boot + Spring Data MongoDB	3.2.x	Flexible document model for complex venue schemas
Booking Service	Spring Boot + Spring Data JPA	3.2.x	ACID transactions critical for payment processing
Notification Service	Spring Boot + Spring AMQP	3.2.x	Asynchronous processing with message broker integration
User DB	PostgreSQL	14+	ACID compliance, strong authentication support
Booking DB	PostgreSQL	14+	Transactional integrity for financial data
Event DB	MongoDB	6.x	Flexible schema for nested seat maps, horizontal scaling
Notification DB	MongoDB	6.x	Document-based logs, high write throughput
Cache Layer	Redis	7.x	Sub-millisecond reads, native TTL support for reservations
Message Broker	RabbitMQ	3.12.x	Reliable message delivery, dead letter queues
Authentication	JSON Web Token (JWT)	jjwt 0.12.x	Stateless authentication, industry standard
Email Service	SendGrid API	Latest	99% deliverability, template support
SMS Service	Twilio API	Latest	Global coverage, East Africa support
Payment Gateway	MTN MoMo API, Airtel Money API	Latest	Dominant mobile money providers in Rwanda
PDF Generation	Apache PDFBox	3.x	Open-source, QR code support
Container Runtime	Docker	24.x	Service isolation, consistent environments

Component	Technology	Version	Justification
Orchestration	Docker Compose (dev), Kubernetes (prod)	Latest	Local development and production scaling
Frontend	React	18.x	Component-based UI, large ecosystem
Monitoring (Optional)	Prometheus + Grafana	Latest	Metrics collection and visualization
Logging (Optional)	ELK Stack (Elasticsearch, Logstash, Kibana)	8.x	Centralized log aggregation

8.1 Technology Rationale

Polyglot Persistence Strategy

PostgreSQL for User and Booking Services:

- ACID transactions essential for financial operations
- Strong foreign key constraints prevent orphaned records
- Mature tooling for connection pooling and replication
- Excellent support for UUID primary keys

MongoDB for Event and Notification Services:

- Event Service: Complex nested documents (venue with sections, rows, seats) match document model perfectly; avoid complex joins
- Notification Service: High write volume for logs; flexible schema for different notification types
- Horizontal sharding support for large-scale event catalogs

Redis for Reservation State

- Native TTL (Time To Live) expiration ideal for 5-minute reservation window
- Key expiry events trigger automatic seat release without polling
- 10x faster than database queries for availability checks
- Atomic operations prevent race conditions in high-concurrency scenarios

RabbitMQ for Asynchronous Messaging

- Guaranteed message delivery with acknowledgment patterns
- Dead letter queues for failed notification retries
- Topic exchanges for event routing (booking.confirmed, user.login)

- Lower operational complexity than Kafka for this scale
-

9. System Constraints and Assumptions

9.1 Constraints

CON-001: Seat reservation timeout is fixed at 5 minutes and shall not be configurable per event or user tier.

CON-002: The system does not implement distributed transaction coordination (e.g., two-phase commit) across microservices; compensating transactions handle failures.

CON-003: Mobile money payment confirmation relies on external gateway callback reliability; the system cannot guarantee real-time payment status.

CON-004: MongoDB document size limit of 16MB restricts maximum event capacity to approximately 50,000 seats per event document.

CON-005: JWT token refresh is not supported; users must re-authenticate after 24-hour expiration.

CON-006: The system does not support partial seat releases; entire reservation (all seats in booking) must be confirmed or released atomically.

9.2 Assumptions

ASSUME-001: PostgreSQL database instances provide query response times under 50ms for indexed lookups under normal load.

ASSUME-002: Network latency between microservices within the same data center does not exceed 10ms.

ASSUME-003: Redis cache maintains 99.9% availability with master-replica configuration for high availability.

ASSUME-004: Mobile Money payment gateways send callback responses within 30 seconds of transaction completion.

ASSUME-005: SendGrid and Twilio APIs maintain 99% uptime for email and SMS delivery.

ASSUME-006: Users have stable internet connectivity during the 5-minute reservation window.

ASSUME-007: Event administrators configure accurate venue layouts with valid seat numbering before publishing events.

ASSUME-008: The target deployment environment supports Docker containerization and has minimum 8GB RAM per microservice instance.

ASSUME-009: Users access the system primarily through modern web browsers (Chrome, Firefox, Safari) with JavaScript enabled.

ASSUME-010: The system operates in a single geographic region (East Africa) with consistent timezone (EAT, UTC+3).

9.3 Dependencies

DEP-001: External payment gateway APIs (MTN MoMo, Airtel Money) must be available for booking confirmation.

DEP-002: Third-party email (SendGrid) and SMS (Twilio) services must be operational for notification delivery.

DEP-003: Frontend application must correctly implement JWT token storage and transmission in request headers.

DEP-004: API Gateway must be running for all client requests; services are not directly accessible from public internet.

DEP-005: Message broker (RabbitMQ) must be running for asynchronous notification delivery; booking operations depend on it.

10. Interface Requirements

10.1 External System Interfaces

10.1.1 Mobile Money Payment Gateway Interface

Interface Type: RESTful HTTPS API

MTN Mobile Money API:

- **Endpoint:** `https://api.mtn.com/collection/v1_0/requesttopay`
- **Authentication:** OAuth 2.0 with API key and secret
- **Request Format:** JSON
- **Response Format:** JSON
- **Timeout:** 30 seconds
- **Retry Policy:** 3 attempts with 5-second intervals

Expected Request:

```
json
```

```
{
  "amount": "50000",
  "currency": "RWF",
  "externalId": "{bookingId}",
  "payer": {
    "partyIdType": "MSISDN",
    "partyId": "250788123456"
  },
  "payerMessage": "Payment for {eventName}",
  "payeeNote": "Booking {confirmationCode}"
}
```

Expected Callback:

```
json

{
  "financialTransactionId": "gateway-tx-12345",
  "externalId": "{bookingId}",
  "status": "SUCCESSFUL",
  "amount": "50000",
  "currency": "RWF"
}
```

10.1.2 Email Delivery Interface (SendGrid)

Interface Type: RESTful HTTPS API

- **Endpoint:** <https://api.sendgrid.com/v3/mail/send>
- **Authentication:** API Key in header
- **Attachments:** Supported (PDF tickets up to 10MB)
- **Rate Limit:** 100 emails per second
- **Templates:** Dynamic template IDs for booking confirmation, failure, login

10.1.3 SMS Delivery Interface (Twilio)

Interface Type: RESTful HTTPS API

- **Endpoint:** <https://api.twilio.com/2010-04-01/Accounts/{AccountSid}/Messages.json>
- **Authentication:** Basic Auth (Account SID + Auth Token)
- **Message Length:** 160 characters (single SMS), concatenated for longer
- **Delivery Receipts:** Webhook callback for delivery status

10.2 Internal Service Interfaces

All internal service-to-service communication shall use RESTful JSON APIs with the following standards:

10.2.1 Common Headers

```
Authorization: Bearer {jwt-token}  
Content-Type: application/json  
X-Correlation-ID: {uuid}  
X-Service-Name: {calling-service}
```

10.2.2 Standard Response Format

Success Response:

```
json  
  
{  
  "success": true,  
  "data": { ... },  
  "timestamp": "2026-01-22T10:30:00Z"  
}
```

Error Response:

```
json  
  
{  
  "success": false,  
  "error": {  
    "code": "SEAT_NOT_AVAILABLE",  
    "message": "The selected seat is no longer available",  
    "details": {  
      "seatId": "uuid",  
      "currentStatus": "SOLD"  
    }  
  },  
  "timestamp": "2026-01-22T10:30:00Z"  
}
```

10.2.3 Standard HTTP Status Codes

Code	Usage
200 OK	Successful GET, PUT requests
201 Created	Successful POST creating new resource
204 No Content	Successful DELETE
400 Bad Request	Invalid request payload or parameters
401 Unauthorized	Missing or invalid JWT token
403 Forbidden	Valid token but insufficient permissions
404 Not Found	Resource does not exist
409 Conflict	Resource state conflict (e.g., seat already locked)
422 Unprocessable Entity	Validation errors
500 Internal Server Error	Unexpected server error
503 Service Unavailable	Downstream dependency failure

11. Data Migration and Seeding

11.1 Initial Data Requirements

DM-001: The system shall include database seeding scripts to create:

- Default ADMIN user with credentials specified in deployment documentation
- Sample event with 100-seat venue for testing purposes
- Notification templates for all notification types

DM-002: Production deployment shall execute database initialization scripts in this order:

1. User Service: Create tables, create admin user
2. Event Service: Create indexes, seed venue templates
3. Booking Service: Create tables, create foreign key constraints
4. Notification Service: Create indexes, seed email/SMS templates

11.2 Backup and Recovery

DM-003: PostgreSQL databases shall be backed up using point-in-time recovery (PITR) with 30-day retention.

DM-004: MongoDB databases shall use replica sets with automatic failover for high availability.

DM-005: Redis data shall not be backed up; reservation state is ephemeral and reconstructible.

12. Testing Requirements

12.1 Unit Testing

TEST-UNIT-001: All service business logic classes shall have unit test coverage exceeding 70%.

TEST-UNIT-002: Database repository methods shall be tested using in-memory databases (H2 for PostgreSQL, Embedded MongoDB).

TEST-UNIT-003: Mock external dependencies (payment gateway, email/SMS APIs) in unit tests.

12.2 Integration Testing

TEST-INT-001: API endpoints shall be tested using Spring Boot's `@WebMvcTest` with mocked service layers.

TEST-INT-002: Database integration tests shall use Testcontainers for PostgreSQL and MongoDB.

TEST-INT-003: Message broker integration shall be tested with embedded RabbitMQ or Testcontainers.

12.3 End-to-End Testing

TEST-E2E-001: Critical user flows shall be tested with automated browser tests (Selenium/Playwright):

- User registration and login
- Complete booking flow (reserve → payment → confirmation)
- Reservation timeout and seat release

TEST-E2E-002: Load testing shall simulate 500 concurrent users attempting to book seats for the same event.

TEST-E2E-003: Chaos engineering tests shall verify system behavior when individual services fail.

12.4 Security Testing

TEST-SEC-001: Penetration testing shall verify JWT token validation prevents unauthorized access.

TEST-SEC-002: SQL injection and NoSQL injection tests shall be performed on all input parameters.

TEST-SEC-003: Payment callback signature validation shall be tested with forged requests.

13. Deployment Architecture

13.1 Development Environment

- **Local Development:** Docker Compose with all services running on single machine
- **Database Persistence:** Docker volumes for data retention across restarts
- **Hot Reload:** Spring Boot DevTools for code changes without restart
- **API Documentation:** Swagger UI accessible at <http://localhost:8080/swagger-ui.html>

13.2 Production Environment (Recommended)

Platform: Kubernetes cluster (AWS EKS, Google GKE, or Azure AKS)

Service Deployment:

- Each microservice: 2-3 replicas for high availability
- API Gateway: 3 replicas with load balancer
- PostgreSQL: Managed database service (AWS RDS, Cloud SQL)
- MongoDB: MongoDB Atlas cluster with replica set
- Redis: Managed Redis (AWS ElastiCache, Google Memorystore)
- RabbitMQ: Managed message broker or 3-node cluster

Networking:

- Ingress controller for external traffic (NGINX Ingress)
- Service mesh (optional): Istio for advanced traffic management
- Internal service-to-service: Kubernetes ClusterIP services

Monitoring:

- Prometheus for metrics collection from all services
 - Grafana dashboards for visualization
 - ELK Stack for centralized logging
 - Jaeger or Zipkin for distributed tracing
-

14. Future Enhancements (Phase 2 Roadmap)

The following capabilities are planned for subsequent development phases and are not included in the current project scope.

14.1 Advanced Payment Features

FE-PAY-001: Support for credit/debit card payments via Stripe or Flutterwave integration.

FE-PAY-002: Split payment functionality allowing multiple users to contribute to single booking.

FE-PAY-003: Installment payment plans for high-value tickets.

FE-PAY-004: Automated refund processing for event cancellations.

14.2 Enhanced User Experience

FE-UX-001: Real-time seat map updates using WebSocket connections to show live availability changes.

FE-UX-002: Seat selection heatmaps showing popular sections based on historical booking data.

FE-UX-003: Accessibility features including screen reader support and keyboard navigation for seat selection.

FE-UX-004: Mobile native applications (iOS, Android) for improved mobile user experience.

14.3 Business Intelligence and Analytics

FE-BI-001: Admin dashboard with event performance metrics (sales velocity, revenue, popular sections).

FE-BI-002: User behavior analytics to track conversion rates and abandonment points.

FE-BI-003: Predictive analytics for demand forecasting and dynamic pricing recommendations.

FE-BI-004: Fraud detection system analyzing booking patterns for suspicious activity.

14.4 Advanced Notification Features

FE-NOT-001: Push notifications to mobile devices for booking reminders and event updates.

FE-NOT-002: Waiting list functionality with automatic notification when sold-out tickets become available.

FE-NOT-003: Event reminder notifications 24 hours before event start time.

FE-NOT-004: Social media sharing integration for event discovery.

14.5 Scalability Enhancements

FE-SCALE-001: Polyglot microservices implementation (NestJS, Python Flask) to demonstrate cross-platform integration.

FE-SCALE-002: Optimistic locking with conflict resolution UI as alternative to pessimistic locking.

FE-SCALE-003: Read replicas for Event Service to handle high-traffic event browsing.

FE-SCALE-004: CDN integration for static event images and venue maps.

14.6 Advanced Security

FE-SEC-001: Multi-factor authentication (MFA) for high-value bookings.

FE-SEC-002: OAuth 2.0 social login integration (Google, Facebook).

FE-SEC-003: Blockchain-based ticket verification to prevent counterfeit tickets.

FE-SEC-004: GDPR compliance features for data export and deletion requests.

15. Glossary of Terms

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability - properties of database transactions
Booking	Complete transaction record including reservation, payment, and confirmation
Confirmation Code	Unique alphanumeric identifier for verified booking
EAT	East Africa Time (UTC+3 timezone)
JWT	JSON Web Token - stateless authentication token standard
Lock	Temporary seat reservation with timeout, preventing other users from selecting
Mobile Money	Electronic payment system using mobile phone accounts (MTN MoMo, Airtel Money)
Pessimistic Locking	Database locking strategy preventing concurrent modifications
Polyglot Persistence	Using different database technologies optimized for specific service needs
QR Code	Quick Response code - 2D barcode containing booking verification data
Reservation	Initial seat hold with countdown timer before payment confirmation
RWF	Rwandan Franc - official currency of Rwanda
Seat Map	Visual representation of venue layout showing seat locations and availability
TTL	Time To Live - automatic expiration mechanism for cached data
USSD	Unstructured Supplementary Service Data - mobile interface for payment prompts

16. Appendices

Appendix A: Sample API Request/Response Examples

A.1 Reserve Seat Request

Request:

http

POST /api/bookings/reserve HTTP/1.1

Host: api.seatwise.rw

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

Content-Type: application/json

```
{
  "eventId": "550e8400-e29b-41d4-a716-446655440000",
  "seatIds": [
    "660e8400-e29b-41d4-a716-446655440001",
    "660e8400-e29b-41d4-a716-446655440002"
  ],
  "phoneNumber": "250788123456"
}
```

Response:

http

HTTP/1.1 201 Created

Content-Type: application/json

```
{
  "success": true,
  "data": {
    "bookingId": "770e8400-e29b-41d4-a716-446655440003",
    "status": "PENDING",
    "totalAmount": 100000,
    "currency": "RWF",
    "expiresAt": "2026-01-22T10:35:00Z",
    "expiresInSeconds": 300,
    "paymentInstructions": {
      "method": "MOBILE_MONEY",
      "message": "Dial *182*7*1# and follow prompts to complete payment"
    }
  },
  "timestamp": "2026-01-22T10:30:00Z"
}
```

A.2 Payment Confirmation Request

Request:

http

POST /api/bookings/770e8400-e29b-41d4-a716-446655440003/payment **HTTP/1.1**

Host: api.seatwise.rw

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

Content-Type: application/json

```
{  
  "phoneNumber": "250788123456",  
  "paymentMethod": "MTN_MOMO"  
}
```

Response:

http

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "success": true,  
  "data": {  
    "bookingId": "770e8400-e29b-41d4-a716-446655440003",  
    "status": "PAYMENT_INITIATED",  
    "transactionId": "880e8400-e29b-41d4-a716-446655440004",  
    "message": "Check your phone for USSD prompt to complete payment"  
  },  
  "timestamp": "2026-01-22T10:31:00Z"  
}
```

A.3 Event Creation Request (Admin)

Request:

http

POST /api/events HTTP/1.1
Host: api.seatwise.rw
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

```
{
  "name": "Kigali Music Festival 2026",
  "title": "Annual Music Festival",
  "description": "Three-day music festival featuring local and international artists",
  "date": "2026-07-15",
  "time": "18:00",
  "venue": {
    "name": "Kigali Arena",
    "address": "KN 4 Ave, Kigali",
    "city": "Kigali",
    "capacity": 10000,
    "layoutType": "stadium"
  },
  "sections": [
    {
      "name": "VIP",
      "price": 50000,
      "rows": 5,
      "seatsPerRow": 20
    },
    {
      "name": "General Admission",
      "price": 20000,
      "rows": 50,
      "seatsPerRow": 100
    }
  ]
}
```

Response:

http

HTTP/1.1 201 Created

Content-Type: application/json

```
{
  "success": true,
  "data": {
    "eventId": "990e8400-e29b-41d4-a716-446655440005",
    "name": "Kigali Music Festival 2026",
    "status": "DRAFT",
    "totalSeats": 5100,
    "availableSeats": 5100,
    "sections": [
      {
        "sectionId": "aa0e8400-e29b-41d4-a716-446655440006",
        "name": "VIP",
        "totalSeats": 100,
        "price": 50000
      },
      {
        "sectionId": "bb0e8400-e29b-41d4-a716-446655440007",
        "name": "General Admission",
        "totalSeats": 5000,
        "price": 20000
      }
    ],
    "createdAt": "2026-01-22T10:32:00Z"
  },
  "timestamp": "2026-01-22T10:32:00Z"
}
```

Appendix B: Deployment Configuration Example

B.1 Docker Compose Configuration (Development)

yaml

version: '3.8'

services:

api-gateway:

image: seatwise/api-gateway:latest

ports:

- "8080:8080"

environment:

- SPRING_PROFILES_ACTIVE=dev
- USER_SERVICE_URL=http://user-service:8081
- EVENT_SERVICE_URL=http://event-service:8082
- BOOKING_SERVICE_URL=http://booking-service:8083
- NOTIFICATION_SERVICE_URL=http://notification-service:8084
- JWT_SECRET=\${JWT_SECRET}

depends_on:

- user-service
- event-service
- booking-service
- notification-service

networks:

- seatwise-network

user-service:

image: seatwise/user-service:latest

ports:

- "8081:8081"

environment:

- SPRING_PROFILES_ACTIVE=dev
- POSTGRES_HOST=user-db
- POSTGRES_PORT=5432
- POSTGRES_DB=seatwise_users
- POSTGRES_USER=seatwise
- POSTGRES_PASSWORD=\${POSTGRES_PASSWORD}
- JWT_SECRET=\${JWT_SECRET}
- RABBITMQ_HOST=rabbitmq

depends_on:

- user-db
- rabbitmq

networks:

- seatwise-network

user-db:

image: postgres:14-alpine

environment:

- POSTGRES_DB=seatwise_users
- POSTGRES_USER=seatwise

- POSTGRES_PASSWORD=\${POSTGRES_PASSWORD}

volumes:

- user-db-data:/var/lib/postgresql/data

networks:

- seatwise-network

event-service:

image: seatwise/event-service:latest

ports:

- "8082:8082"

environment:

- SPRING_PROFILES_ACTIVE=dev

- MONGO_URI=mongodb://event-db:27017/seatwise_events

- REDIS_HOST=redis

- REDIS_PORT=6379

depends_on:

- event-db

- redis

networks:

- seatwise-network

event-db:

image: mongo:6

environment:

- MONGO_INITDB_DATABASE=seatwise_events

volumes:

- event-db-data:/data/db

networks:

- seatwise-network

booking-service:

image: seatwise/booking-service:latest

ports:

- "8083:8083"

environment:

- SPRING_PROFILES_ACTIVE=dev

- POSTGRES_HOST=booking-db

- POSTGRES_PORT=5432

- POSTGRES_DB=seatwise_bookings

- POSTGRES_USER=seatwise

- POSTGRES_PASSWORD=\${POSTGRES_PASSWORD}

- REDIS_HOST=redis

- REDIS_PORT=6379

- RABBITMQ_HOST=rabbitmq

- EVENT_SERVICE_URL=http://event-service:8082

- MTN_MOMO_API_KEY=\${MTN_MOMO_API_KEY}

- AIRTEL_MONEY_API_KEY=\${AIRTEL_MONEY_API_KEY}

depends_on:

- booking-db
- redis
- rabbitmq
- event-service

networks:

- seatwise-network

booking-db:

image: postgres:14-alpine

environment:

- POSTGRES_DB=seatwise_bookings
- POSTGRES_USER=seatwise
- POSTGRES_PASSWORD=\${POSTGRES_PASSWORD}

volumes:

- booking-db-data:/var/lib/postgresql/data

networks:

- seatwise-network

notification-service:

image: seatwise/notification-service:latest

ports:

- "8084:8084"

environment:

- SPRING_PROFILES_ACTIVE=dev
- MONGO_URI=mongodb://notification-db:27017/seatwise_notifications
- REDIS_HOST=redis
- REDIS_PORT=6379
- RABBITMQ_HOST=rabbitmq
- SENDGRID_API_KEY=\${SENDGRID_API_KEY}
- TWILIO_ACCOUNT_SID=\${TWILIO_ACCOUNT_SID}
- TWILIO_AUTH_TOKEN=\${TWILIO_AUTH_TOKEN}

depends_on:

- notification-db
- redis
- rabbitmq

networks:

- seatwise-network

notification-db:

image: mongo:6

environment:

- MONGO_INITDB_DATABASE=seatwise_notifications

volumes:

- notification-db-data:/data/db

networks:

- seatwise-network

redis:

image: redis:7-alpine

ports:

- "6379:6379"

command: redis-server --appendonly yes

volumes:

- redis-data:/data

networks:

- seatwise-network

rabbitmq:

image: rabbitmq:3.12-management-alpine

ports:

- "5672:5672"

- "15672:15672"

environment:

- RABBITMQ_DEFAULT_USER=seatwise

- RABBITMQ_DEFAULT_PASS=\${RABBITMQ_PASSWORD}

volumes:

- rabbitmq-data:/var/lib/rabbitmq

networks:

- seatwise-network

frontend:

image: seatwise/frontend:latest

ports:

- "3000:80"

environment:

- REACT_APP_API_URL=http://localhost:8080/api

depends_on:

- api-gateway

networks:

- seatwise-network

volumes:

user-db-data:

booking-db-data:

event-db-data:

notification-db-data:

redis-data:

rabbitmq-data:

networks:

seatwise-network:

driver: bridge

B.2 Environment Variables Template

```
# JWT Configuration
JWT_SECRET=your-256-bit-secret-key-here-change-in-production
ADMIN_SECRET_KEY=your-admin-registration-secret-key

# Database Passwords
POSTGRES_PASSWORD=secure_postgres_password
RABBITMQ_PASSWORD=secure_rabbitmq_password

# Payment Gateway API Keys
MTN_MOMO_API_KEY=your-mtn-momo-api-key
MTN_MOMO_API_SECRET=your-mtn-momo-api-secret
AIRTEL_MONEY_API_KEY=your-airtel-money-api-key
AIRTEL_MONEY_API_SECRET=your-airtel-money-api-secret

# Notification Service API Keys
SENDGRID_API_KEY=your-sendgrid-api-key
TWILIO_ACCOUNT_SID=your-twilio-account-sid
TWILIO_AUTH_TOKEN=your-twilio-auth-token
TWILIO_PHONE_NUMBER=your-twilio-phone-number
```

Appendix C: Monitoring Dashboard Metrics

C.1 Key Performance Indicators (KPIs)

Service Health Metrics:

- Request rate (requests/second) per service
- Error rate (%) with breakdown by HTTP status code
- Response time (p50, p95, p99 percentiles)
- Service uptime (%) over 24-hour and 7-day periods
- Active concurrent connections per service

Business Metrics:

- Active reservations count (LOCKED seats)
- Bookings confirmed per hour
- Revenue per event and total platform revenue
- Seat utilization rate (%) per event
- Average time to confirm booking (seconds)
- Reservation abandonment rate (%)
- Conversion rate (reserved → confirmed)

- Failed payment rate (%)

Infrastructure Metrics:

- CPU utilization (%) per service container
- Memory usage (MB) and heap size
- Database connection pool utilization
- Redis cache hit rate (%)
- Redis cache miss rate (%)
- Message queue depth (pending messages)
- Disk I/O operations per second
- Network throughput (Mbps)

User Experience Metrics:

- Average page load time
- Time to first seat lock (ms)
- Payment callback response time
- Notification delivery success rate (%)
- PDF ticket generation time (ms)

C.2 Alert Thresholds

Metric	Warning Threshold	Critical Threshold
API Gateway Error Rate	> 2%	> 5%
Service Response Time (p95)	> 1000ms	> 2000ms
Database Connection Pool Usage	> 80%	> 95%
Redis Cache Hit Rate	< 85%	< 70%
RabbitMQ Queue Depth	> 1000 messages	> 5000 messages
Active Expired Reservations	> 100	> 500
CPU Usage	> 70%	> 90%
Memory Usage	> 75%	> 90%
Failed Notification Delivery Rate	> 5%	> 15%

Appendix D: Database Initialization Scripts

D.1 User Service Schema (PostgreSQL)

sql

-- Create users table

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  username VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  full_name VARCHAR(500) NOT NULL,  
  phone_number VARCHAR(20),  
  role VARCHAR(20) NOT NULL CHECK (role IN ('USER', 'ADMIN')),  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  last_login TIMESTAMP  
);
```

-- Create indexes

```
CREATE INDEX idx_users_username ON users(username);  
CREATE INDEX idx_users_role ON users(role);  
CREATE INDEX idx_users_created_at ON users(created_at);
```

-- Create function to update updated_at timestamp

```
CREATE OR REPLACE FUNCTION update_updated_at_column()  
RETURNS TRIGGER AS $  
BEGIN  
  NEW.updated_at = CURRENT_TIMESTAMP;  
  RETURN NEW;  
END;  
$ language 'plpgsql';
```

-- Create trigger for updated_at

```
CREATE TRIGGER update_users_updated_at  
BEFORE UPDATE ON users  
FOR EACH ROW  
EXECUTE FUNCTION update_updated_at_column();
```

-- Insert default admin user (password: Admin@123 - CHANGE IN PRODUCTION)

```
INSERT INTO users (username, password_hash, full_name, role)  
VALUES (  
  'admin@seatwise.rw',  
  '$2a$12$LQv3c1yqBWVHxkd0LHakCOYz6TtxMQJqhN8/LewY5IW.G5L5gNXRC',  
  'System Administrator',  
  'ADMIN'  
);
```

D.2 Booking Service Schema (PostgreSQL)

sql

-- Create bookings table

```
CREATE TABLE bookings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL,  
  event_id UUID NOT NULL,  
  seat_ids JSONB NOT NULL,  
  total_amount DECIMAL(10,2) NOT NULL,  
  status VARCHAR(30) NOT NULL CHECK (status IN ('PENDING', 'PAYMENT_INITIATED', 'CONFIRMED', 'FAILED')  
  confirmation_code VARCHAR(20) UNIQUE,  
  reserved_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  payment_initiated_at TIMESTAMP,  
  confirmed_at TIMESTAMP,  
  expires_at TIMESTAMP NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

-- Create payment_transactions table

```
CREATE TABLE payment_transactions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,  
  payment_method VARCHAR(50) NOT NULL,  
  phone_number VARCHAR(20) NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  gateway_reference VARCHAR(255),  
  status VARCHAR(30) NOT NULL CHECK (status IN ('INITIATED', 'SUCCESS', 'FAILED')),  
  callback_data JSONB,  
  initiated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  completed_at TIMESTAMP  
);
```

-- Create indexes

```
CREATE INDEX idx_bookings_user_id ON bookings(user_id);  
CREATE INDEX idx_bookings_event_id ON bookings(event_id);  
CREATE INDEX idx_bookings_status ON bookings(status);  
CREATE INDEX idx_bookings_confirmation_code ON bookings(confirmation_code);  
CREATE INDEX idx_bookings_expires_at ON bookings(expires_at) WHERE status = 'PENDING';  
CREATE INDEX idx_bookings_created_at ON bookings(created_at);
```

```
CREATE INDEX idx_transactions_booking_id ON payment_transactions(booking_id);  
CREATE INDEX idx_transactions_status ON payment_transactions(status);  
CREATE INDEX idx_transactions_gateway_ref ON payment_transactions(gateway_reference);
```

-- Create trigger for updated_at

```
CREATE TRIGGER update_bookings_updated_at  
  BEFORE UPDATE ON bookings
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION update_updated_at_column();
```

D.3 Event Service Indexes (MongoDB)

```
javascript
```

```
// Connect to MongoDB
```

```
use seatwise_events;
```

```
// Create indexes for events collection
```

```
db.events.createIndex({ "eventId": 1 }, { unique: true });
```

```
db.events.createIndex({ "date": 1, "status": 1 });
```

```
db.events.createIndex({ "status": 1 });
```

```
db.events.createIndex({ "sections.rows.seats.seatId": 1 });
```

```
db.events.createIndex({ "sections.rows.seats.status": 1 });
```

```
db.events.createIndex({ "createdBy": 1 });
```

```
db.events.createIndex({ "createdAt": -1 });
```

```
// Insert sample event for testing
```

```
db.events.insertOne({  
  "eventId": UUID(),  
  "name": "Sample Concert 2026",  
  "title": "Test Event for Development",  
  "description": "This is a sample event for testing purposes",  
  "date": new Date("2026-06-15"),  
  "time": "19:00",  
  "venue": {  
    "name": "Test Arena",  
    "address": "123 Test Street",  
    "city": "Kigali",  
    "capacity": 100,  
    "layoutType": "theater"  
  },  
  "sections": [  
    {  
      "sectionId": UUID(),  
      "name": "General",  
      "price": 10000,  
      "totalSeats": 100,  
      "availableSeats": 100,  
      "rows": [  
        {  
          "rowLabel": "A",  
          "seats": Array.from({length: 10}, (_, i) => ({  
            "seatId": UUID(),  
            "seatNumber": `A${i+1}`,  
            "status": "AVAILABLE",  
            "lockedBy": null,  
            "lockedAt": null,  
            "bookedBy": null,  
            "bookingId": null  
          }  
        }  
      ]  
    }  
  ]  
})
```

```
        }  
    ]  
}  
],  
"status": "PUBLISHED",  
"createdBy": null,  
"createdAt": new Date(),  
"updatedAt": new Date()  
});
```

D.4 Notification Service Indexes (MongoDB)

javascript

```
// Connect to MongoDB
```

```
use seatwise_notifications;
```

```
// Create indexes for notifications collection
```

```
db.notifications.createIndex({ "notificationId": 1 }, { unique: true });
```

```
db.notifications.createIndex({ "userId": 1, "createdAt": -1 });
```

```
db.notifications.createIndex({ "status": 1, "createdAt": -1 });
```

```
db.notifications.createIndex({ "type": 1 });
```

```
db.notifications.createIndex({ "createdAt": -1 });
```

```
// Create indexes for notification_logs collection
```

```
db.notification_logs.createIndex({ "notificationId": 1 });
```

```
db.notification_logs.createIndex({ "channel": 1, "success": 1 });
```

```
db.notification_logs.createIndex({ "attemptedAt": -1 });
```

```
// Insert notification templates
```

```
db.notification_templates.insertMany([
```

```
{
  "type": "BOOKING_CONFIRMED",
  "emailSubject": "Booking Confirmed - {{eventName}}",
  "emailTemplate": "Your booking for {{eventName}} has been confirmed. Your confirmation code is {{confirmationCode}}",
  "smsTemplate": "Booking confirmed for {{eventName}}. Code: {{confirmationCode}}. Your ticket is attached to the code {{confirmationCode}}",
},
{
  "type": "BOOKING_FAILED",
  "emailSubject": "Booking Failed - {{eventName}}",
  "emailTemplate": "Unfortunately, your booking for {{eventName}} could not be completed. Reason: {{reason}}",
  "smsTemplate": "Booking failed for {{eventName}}. Please try again or contact support.",
},
{
  "type": "PAYMENT_PENDING",
  "emailSubject": "Payment Pending - {{eventName}}",
  "emailTemplate": "Your seat reservation for {{eventName}} is pending payment. Complete payment within {{minutesRemaining}} minutes.",
  "smsTemplate": "Payment pending for {{eventName}}. Complete within {{minutesRemaining}} min or reservation will be cancelled.",
},
{
  "type": "USER_LOGIN",
  "emailSubject": "New Login Detected",
  "emailTemplate": "A new login to your SeatWise account was detected at {{loginTime}} from {{location}}.",
  "smsTemplate": null
}
]);
```

Appendix E: RabbitMQ Queue Configuration

E.1 Exchange and Queue Setup

```
javascript
```

```
// Queue names
```

```
const QUEUES = {  
  BOOKING_CONFIRMED: 'booking.confirmed',  
  BOOKING_FAILED: 'booking.failed',  
  BOOKING_EXPIRED: 'booking.expired',  
  USER_LOGIN: 'user.login'  
};
```

```
// Exchange configuration
```

```
const EXCHANGE = {  
  name: 'seatwise.events',  
  type: 'topic',  
  durable: true  
};
```

```
// Routing keys
```

```
const ROUTING_KEYS = {  
  BOOKING_CONFIRMED: 'booking.confirmed',  
  BOOKING_FAILED: 'booking.failed',  
  BOOKING_EXPIRED: 'booking.expired',  
  USER_LOGIN: 'user.login'  
};
```

```
// Dead letter exchange for failed messages
```

```
const DLX = {  
  name: 'seatwise.dlx',  
  type: 'direct',  
  durable: true  
};
```

17. Approval and Sign-Off

This Software Requirements Specification has been reviewed and approved by the following stakeholders:

Role	Name	Signature	Date
Product Owner			
Technical Lead			
Lead Backend Developer			
QA Lead			
DevOps Lead			
Security Officer			

Document Information

Document Classification: Internal Use
Distribution: Development Team, Project Stakeholders, Academic Reviewers
Next Review Date: March 22, 2026
Version Control: Git repository - (/docs/SRS.md)
Contact for Clarifications: [Project Technical Lead Email]

Revision Notes:

This SRS document integrates the complete system architecture design with comprehensive functional and non-functional requirements. All architectural decisions have been justified with explicit tradeoffs and constraints. The document serves as the authoritative specification for SeatWise Phase 1 development.

Change Management:

Any modifications to this SRS must be:

- 1. Proposed via formal change request process
- 2. Reviewed by Technical Lead and Product Owner
- 3. Impact-analyzed for existing development work
- 4. Approved by majority stakeholder consensus
- 5. Version-controlled with changelog updates

Document Total: 17 Sections, 150+ Requirements, 9 Database Schemas, 25+ API Endpoints

Prepared by: SeatWise Development Team

Last Updated: January 22, 2026

Document Status:  Complete and Ready for Development